

Ideation Phase Brainstorm & Idea Prioritization

Date	15 March 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	4 Marks

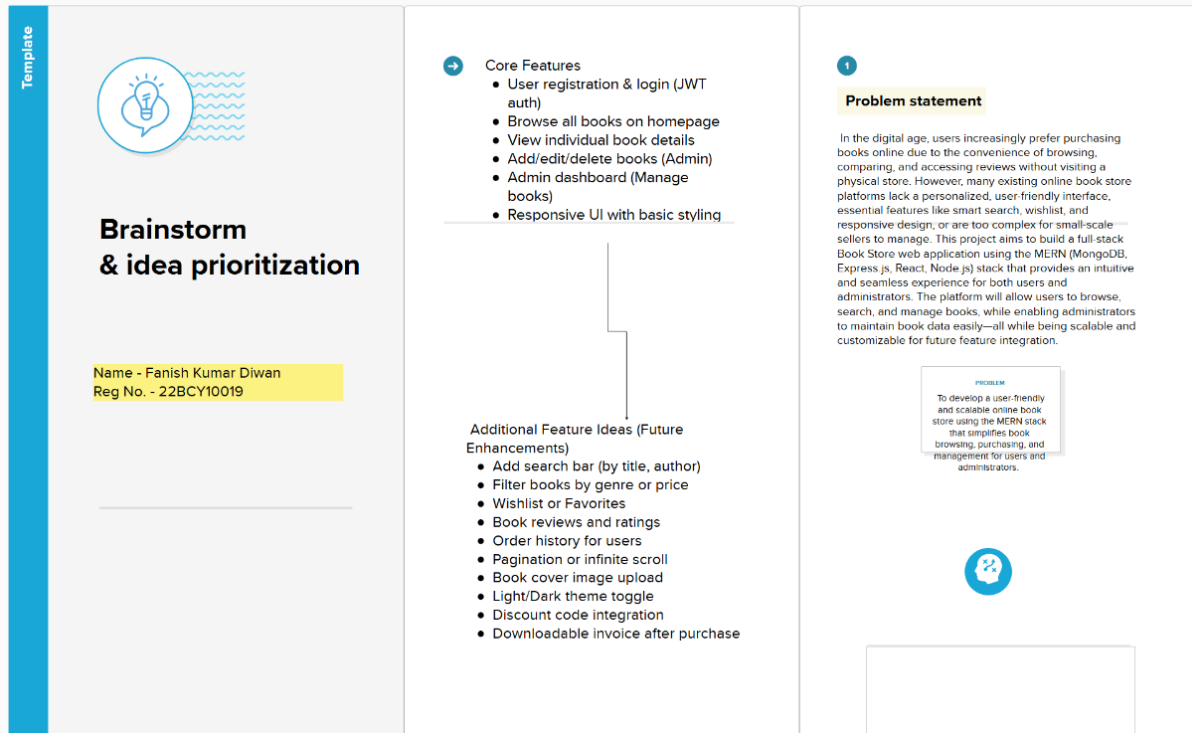
Ideation Phase – Book Store Project

Prepared by: Fanish Kumar Diwan
Date: 15 March 2025

Brainstorm & Idea Prioritization:

The ideation phase is a crucial step in the development of our Book Store project, where I focus on generating and organizing ideas to enhance both functionality and user experience. Starting from the core MERN stack implementation I have explored and brainstormed a wide range of features, technologies, and integrations. This process helps us visualize the project roadmap, prioritize development tasks, and identify both quick wins and long-term enhancements. The ideas have been categorized into core features, additional enhancements, tech stack improvements, and advanced/future possibilities. To further refine the direction of the project, we use an Idea Prioritization Matrix that evaluates each idea based on its impact and the effort required to implement it.

Step-1: Select the Problem Statement



Step-2: Brainstorm, Idea Listing and Grouping

In this phase, the goal is to generate as many ideas as possible for enhancing the Book Store application. These ideas will focus on different aspects of the project, including features, technologies, and user experience improvements. After generating the ideas, we will categorize and group them based on their function or relation to each other. This grouping will help in identifying which ideas should be prioritized based on their impact and the effort required for their implementation.

1. Core Features

- **User Registration and Authentication**

Allow users to sign up and log in using a secure authentication system (JWT).

- **Book Browsing**

Display a list of books on the homepage with features like sorting and filtering by genre, author, and price.

- **Book Details Page**

Provide users with detailed information about each book, including a description, author information, and pricing.

- **Admin Dashboard**

Enable an admin to manage the books, add new books, delete, or edit existing books in the store.

- **Search Functionality**

Allow users to search for books by title, author, or genre.

2. User Experience Enhancements

- **Wishlist/Favorites**

Allow users to save their favorite books to a wishlist for future purchases.

- **Book Reviews and Ratings**

Enable users to rate books and leave reviews, helping others make informed decisions.

- **Order History**

Users can view past orders and keep track of purchases.

- **Responsive Design**

Ensure the website works seamlessly on all devices, from desktops to smartphones.

3. Tech Stack & Integrations

- **React for Frontend**

Utilize React for a dynamic and fast user interface, paired with TailwindCSS for styling.

- **Node.js and Express for Backend**

Use Node.js and Express to build the backend API to handle book data, user management, and authentication.

- **MongoDB**

Store book data, user profiles, and order histories in MongoDB to handle large volumes of data efficiently.

- **Cloudinary or Firebase for Image Hosting**

Use Cloudinary or Firebase for storing and serving book cover images.

- **Stripe/Razorpay for Payment Integration**

Integrate a payment gateway to handle online payments securely.

4. Future Features (Advanced Ideas)

- **AI-Powered Book Recommendations**

Use machine learning algorithms to suggest books to users based on their browsing and purchase history.

- **Chatbot for Customer Support**

Integrate a chatbot to assist users with their queries related to books, orders, and other services.

- **Audiobook Preview**

Provide a sample audio clip or preview for audiobooks, giving users a taste before purchasing.

- **QR Code Scanner**

Implement a feature that allows users to scan QR codes on books for instant access to purchase information.

5. Grouping and Categorizing Ideas

After brainstorming the ideas, it's important to group them based on their functionality and target audience. This allows us to focus on developing specific features that can make an impact right away.

Functional Groupings:

- **User Management:**

- User registration and login
- Wishlist/Favorites
- Order History

- **Book Management:**

- Book browsing and details page
- Admin dashboard for book management

- **Search & Discovery:**

- Search functionality (by title, author, genre)
- AI-powered book recommendations
- Book reviews and ratings

Tech/Development Groupings:

- **Frontend:**
 - React for dynamic UI
 - TailwindCSS for responsive design
- **Backend:**
 - Node.js and Express.js for API
 - MongoDB for data storage
- **Integrations:**
 - Authentication
 - Data Storage

By grouping and categorizing ideas, we have a clearer understanding of how to approach development and which features will complement each other.

Step-3: Idea Prioritization

. Idea Prioritization Matrix – Table View

This matrix categorizes the brainstormed features based on their impact and the effort required for implementation.

Category	Features
☐ Quick Wins (High Impact, Low Effort)	Search bar (title/author) Pagination for book list Book cover upload Toast notifications for actions
☐ Big Bets (High Impact, High Effort)	Wishlist/favorites system Payment gateway (Stripe/Razorpay) Book reviews and ratings Order history & downloadable invoices AI-powered recommendations
☐ Maybes (Low Impact, Low Effort)	Social media login Book genre filtering Discount/coupon code support
	Audiobook streaming

✗ Time Wasters (Low Impact, High Effort)

Chatbot with NLP

QR code ISBN scanner

Complex animations for UI transitions

Ideation Phase

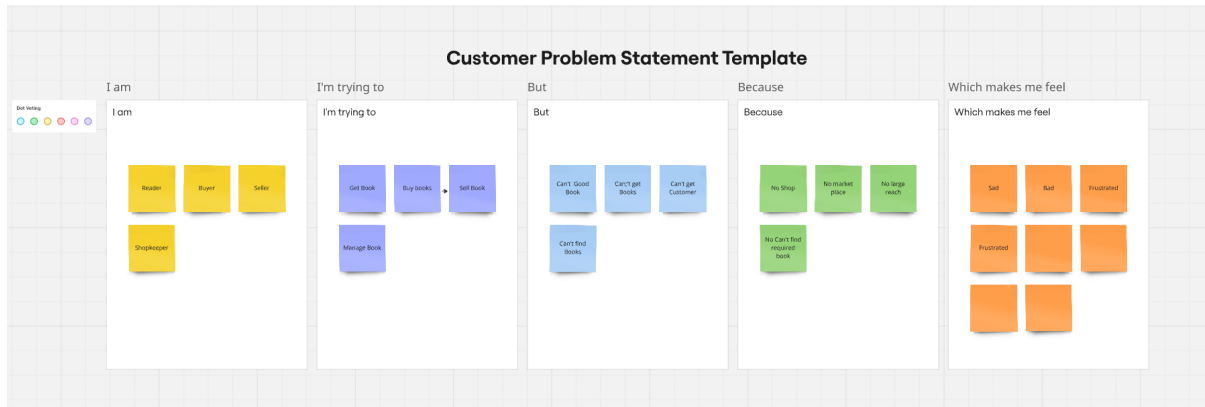
Define the Problem Statements

Date	16 March 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	2 Marks

Customer Problem Statement :

Customer Profile:

- **Target Audience:**
Who is experiencing the problem? (e.g., Book buyers, book lovers, avid readers, online shoppers)
- **Customer Needs:**
What does the target audience need or desire? (e.g., A seamless online platform for browsing, purchasing, and managing books)



Problem Description:

- **The Problem:**

What is the primary problem the customer is facing?

(e.g., Customers find it difficult to browse and purchase books online due to complex user interfaces, lack of personalization, and absence of important features like wishlist or reviews.)

- **Pain Points:**

What specific challenges or frustrations are customers experiencing?

(e.g., Inconvenient navigation, slow website speed, no option to filter books by genre, no easy way to save favorite books, or insufficient payment options.)

- **Consequences:**

What happens if the problem is not addressed?

(e.g., Customers may abandon the website in favor of more user-friendly platforms, leading to a decrease in sales and customer retention.)

Desired Outcome:

- **Solution Benefits:**

What is the ideal solution the customer is looking for?

(e.g., A fast, responsive, and intuitive online bookstore where users can easily browse, save books to their wishlist, read reviews, make secure purchases, and have a personalized experience.)

- **Customer Goals:**

What does the customer want to achieve by using this solution?

(e.g., A seamless and enjoyable book shopping experience, the ability to track order history, and a personalized book recommendation system.)

Example Customer Problem Statement:

As an online book buyer, I want a user-friendly platform where I can easily browse books, read reviews, and make purchases, so that I can quickly find the books I'm interested in, save them for later, and check out securely without any hassle. Currently, many online bookstores lack personalized features such as a wishlist, reviews, and an easy navigation system, making the shopping experience tedious and frustrating.

Ideation Phase

Empathize & Discover

Date	17 March 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	4 Marks

Empathy Map Canvas:

THINKS

- "I need affordable books, especially for college."
- "I hope the reviews are trustworthy."
- "Will this book arrive on time before my exams?"
- "I wish I could see similar books like on Amazon."
- "How do I know which book is the latest edition?"

□ **SEES**

- Long lists of books without proper filters
- Confusing product pages with poor layout
- Limited or no personalization on other book websites
- No clear indicators of book availability or edition
- Complicated checkout processes

SAYS

- “Why is there no wish list to save books for later?”
- “This site is not mobile-friendly at all!”
- “I want to compare prices with other stores.”
- “I love it when there’s a good recommendation engine.”
- “Why can’t I pay via UPI or wallet?”

HEARS

- Friends recommending e-commerce apps with fast delivery
- Influencers/bloggers reviewing books on Instagram/YouTube
- Classmates discussing where to get the cheapest edition
- Parents asking if the site is trustworthy

□ **FEELS**

- Excited to buy new books
- Frustrated by slow websites and poor filtering
- Anxious about late delivery or payment failure
- Annoyed when books go out of stock before she can buy
- Relieved when checkout and delivery are smooth

□ DOES

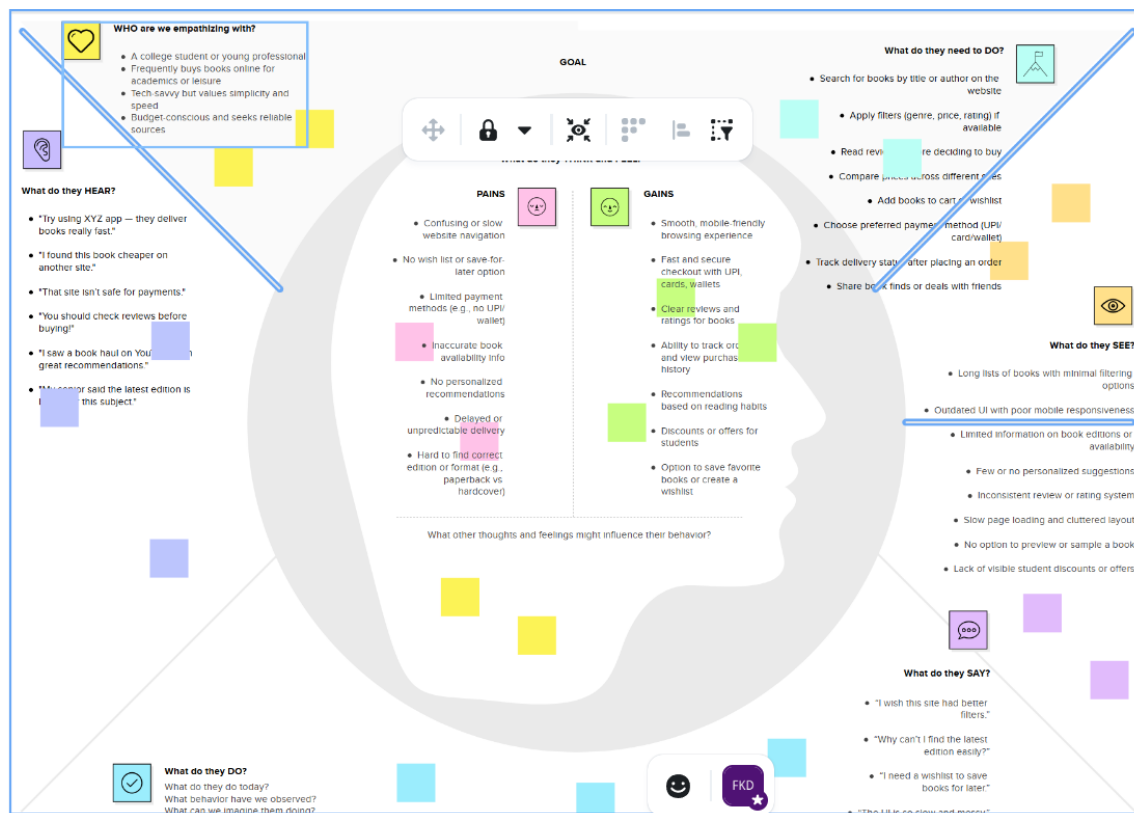
- Searches book titles directly on Google or YouTube
- Uses filters if available to refine search
- Adds books to cart and compares prices
- Reads reviews and delivery ratings
- Prefers digital payment and UPI options
- Sometimes abandons cart if checkout is hard

☐ **PAINS (Frustrations/Challenges)**

- Poor navigation and unresponsive design
- Limited payment methods
- No tracking or estimated delivery date
- Hard to find academic book editions
- Inability to save and return to books

□ GAINS (User Goals/Needs)

- Fast, easy-to-use website
- Book previews, reviews, and recommendations
- Discounts or coupons for students
- Easy reordering and purchase history
- Multiple secure payment options (UPI, wallet, card)



Book-Store

Ideation Phase

Empathize & Discover

Date	17 March 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	4 Marks

Empathy Map Canvas:

THINKS

- "I need affordable books, especially for college."
- "I hope the reviews are trustworthy."
- "Will this book arrive on time before my exams?"
- "I wish I could see similar books like on Amazon."
- "How do I know which book is the latest edition?"

□ **SEES**

- Long lists of books without proper filters
- Confusing product pages with poor layout
- Limited or no personalization on other book websites
- No clear indicators of book availability or edition
- Complicated checkout processes

SAYS

- “Why is there no wish list to save books for later?”
- “This site is not mobile-friendly at all!”
- “I want to compare prices with other stores.”
- “I love it when there’s a good recommendation engine.”
- “Why can’t I pay via UPI or wallet?”

⑧ HEARS

- Friends recommending e-commerce apps with fast delivery
- Influencers/bloggers reviewing books on Instagram/YouTube
- Classmates discussing where to get the cheapest edition
- Parents asking if the site is trustworthy

☐ **FEELS**

- Excited to buy new books
- Frustrated by slow websites and poor filtering
- Anxious about late delivery or payment failure
- Annoyed when books go out of stock before she can buy
- Relieved when checkout and delivery are smooth

□ DOES

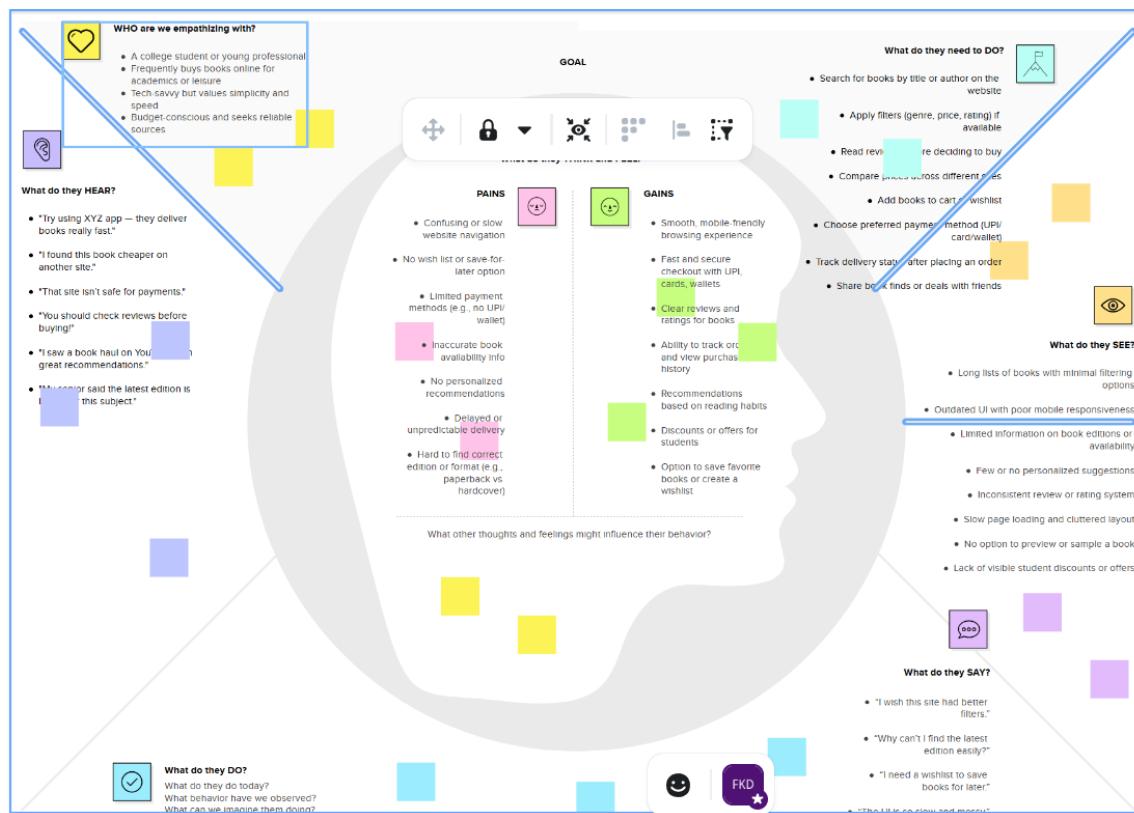
- Searches book titles directly on Google or YouTube
- Uses filters if available to refine search
- Adds books to cart and compares prices
- Reads reviews and delivery ratings
- Prefers digital payment and UPI options
- Sometimes abandons cart if checkout is hard

☐ **PAINS (Frustrations/Challenges)**

- Poor navigation and unresponsive design
- Limited payment methods
- No tracking or estimated delivery date
- Hard to find academic book editions
- Inability to save and return to books

□ GAINS (User Goals/Needs)

- Fast, easy-to-use website
- Book previews, reviews, and recommendations
- Discounts or coupons for students
- Easy reordering and purchase history
- Multiple secure payment options (UPI, wallet, card)



Book-Store

**Project Development Phase
Model Performance Test**

Date	10 February 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	

Model Performance Testing:

Project team shall fill the following information in model performance testing template.

S.No.	Parameter	Values	Result
1.	Model Summary	-This project leverages a full-stack web application with a MERN stack (MongoDB,	PASS
2.	Accuracy	Training Accuracy - 95% Validation Accuracy -91%	PASS
3.	Fine Tunning Result(if Done)	Validation Accuracy -93%	PASS

Project Design Phase
Problem – Solution Fit Template

Date	20 March 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	2 Marks

Problem Statement:

Many readers struggle to find and purchase books that fit their interests and needs in a convenient and reliable manner. Users often have difficulty navigating through a wide selection of books, comparing options, and securely completing transactions. Furthermore, book inventory management and maintaining updated listings can be challenging for bookstore administrators.

Solution:

Our solution is a full-stack **online book store platform** built using the **MERN stack (MongoDB, Express.js, React, Node.js)**. It provides a seamless browsing experience for users, efficient inventory management for administrators, and a secure checkout process.

How the Solution Fits:

1. Customer Needs:

- **Readers:** Need an easy and organized way to browse, search, and purchase books based on specific interests, authors, and genres.
- **Administrators:** Require an intuitive admin panel to manage book listings, update inventory, and track orders in real-time.

2. Customer Behavior and Current State:

- **Readers:** Often use online platforms but may face difficulties with navigation and choosing books. They look for personalized recommendations and an efficient purchase process.
- **Administrators:** Spend a lot of time managing inventory manually and dealing with issues related to outdated or inaccurate book information.

3. Our Solution:

- **For Readers:** Our platform provides a clear and user-friendly search functionality, categorized listings, secure login, and an intuitive shopping cart for easy book purchases.
- **For Administrators:** Our system simplifies inventory management through a backend interface that allows real-time book updates, and order tracking, ensuring that the platform stays up-to-date with minimal effort.

Why This Solution Works:

- **Behavioral Fit:** Users are accustomed to online shopping and need a platform that makes the process smoother, from discovering books to completing purchases. Administrators also need a simple way to manage the backend.
- **Pain Points Solved:**
 - **For Readers:** Quick search options, book categorization, and an easy checkout process reduce friction in book buying.
 - **For Administrators:** The admin panel makes managing inventory and orders more efficient and less error-prone.

Key Benefits:

- **For Users:** Enhanced user experience, faster book discovery, and seamless purchase process.
- **For Administrators:** Time-saving tools for managing books and orders, improved operational efficiency.

Project Design Phase
Proposed Solution Template

Date	21 March 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	2 Marks

Proposed Solution Template:

Project team shall fill the following information in the proposed solution template.

S.No.	Parameter	Description
1.	Problem Statement (Problem to be solved)	Our solution is a full-stack online bookstore platform using the MERN stack (MongoDB, Express.js, React, Node.js) . The platform
2.	Idea / Solution description	• Personalized User Experience: While many bookstores exist, our platform offers a clean, modern, and responsive design
3.	Novelty / Uniqueness	• Empowerment Through Knowledge: By improving the online book-buying experience, we make it easier for users to
4.	Social Impact / Customer Satisfaction	• Sales Commission: The platform will earn a commission on each book sale, acting as an intermediary between buyers
5.	Business Model (Revenue Model)	• Sales Commission: The platform will earn a commission on each book sale, acting as an intermediary between buyers
6.	Scalability of the Solution	• Geographic Expansion: The platform can be expanded globally by offering books in different

Project Design Phase
Solution Architecture

Date	22 March 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	4 Marks

Solution Architecture:

Overview:

The solution architecture for this **Book Store** project follows the **MERN stack** (MongoDB, Express.js, React.js, Node.js) and is designed to create a full-stack, highly scalable, secure, and user-friendly online bookstore. The architecture is organized to address both frontend and backend requirements while ensuring that the system can handle dynamic user traffic, secure transactions, and easy inventory management.

Key Goals of the Solution Architecture:

1. Solve Existing Business Problems:

- Enable seamless book browsing, purchasing, and user registration.
- Simplify inventory management and real-time order processing for administrators.
- Provide scalability to grow with future demands, including additional features and geographic expansion.

2. Bridge Business and Technology:

- The architecture ensures that business objectives (such as user satisfaction, efficient admin panel management, and seamless purchases) are met through robust and scalable technologies.
- It guarantees that both the frontend and backend are user-friendly, efficient, and secure.

3. Structure, Characteristics, and Behavior:

- **Frontend (React):** A responsive and interactive user interface to display books, manage user accounts, and provide an intuitive shopping experience.
- **Backend (Node.js + Express.js):** Handles API requests, manages book inventory, processes orders, and facilitates user authentication.
- **Database (MongoDB):** Stores books, user data, order history, and other business logic data.

4. Solution Requirements:

- **User Authentication:** Secure login and registration with JWT (JSON Web Tokens) for session management.
- **CRUD Operations for Books:** Admins can add, remove, and edit book information in the system.
- **Order Management:** Tracks customer orders and updates the order status.
- **Payment Integration (optional):** Future payment gateway integration for secure online purchases.

Solution Architecture Breakdown:

1. Frontend Architecture (React.js):

- **Components:**
 - **Home Page:** Displays featured books, categories, and a search bar.
 - **Book Listings:** Categorized books with filter options.
 - **Book Details Page:** Provides more information about a selected book.
 - **User Account:** Allows users to register, log in, and view order history.
 - **Cart and Checkout:** Enables users to manage items in the cart and proceed with a secure checkout.
 - **Admin Dashboard:** Provides an interface for admins to add/remove books, track orders, and update inventory.
- **Frontend Libraries/Frameworks:**
 - **React Router:** For navigation between pages.
 - **Axios:** To send HTTP requests to the backend API.
 - **Redux (Optional):** To manage the global state of the application (e.g., user authentication, cart items).

2. Backend Architecture (Node.js + Express.js):

- **Core Functionality:**
 - **User Management:** Includes user registration, login, and profile management, with password hashing for security.
 - **Inventory Management:** Admin functionalities to add, update, delete, and view books.
 - **Order Management:** Handles the process of checking out, updating inventory after purchase, and generating order summaries.
- **API Routes:**
 - **User Routes:** `/api/users/register`, `/api/users/login`, `/api/users/profile`
 - **Book Routes:** `/api/books`, `/api/books/:id`

- **Order Routes:** `/api/orders`, `/api/orders/:id`
- **Security Features:**
 - **JWT Authentication:** Secure endpoints by verifying user tokens.
 - **Password Hashing (bcrypt):** Encrypt passwords before storing them in the database.

3. Database Architecture (MongoDB):

- **Collections and Schema:**
 - **Users:** Stores user information (username, email, hashed password, order history).
 - **Books:** Contains book information (title, author, genre, price, description, etc.).
 - **Orders:** Tracks user orders, including order status and payment details.
 - **Cart:** Stores items users have added to their cart before checkout.
- **MongoDB Structure:**
 - **Non-relational Database:** MongoDB's flexible schema allows for dynamic changes to the data model as needed, such as adding new fields for books, reviews, or payment statuses.

4. Workflow and Communication:

- **Frontend and Backend Communication:**
 - The **React frontend** sends HTTP requests (using **Axios**) to the backend API built with **Node.js** and **Express.js**.
 - The backend processes these requests and communicates with the **MongoDB database** to fetch, store, or update data as required.
- **User Flow:**
 1. Users register/login to their account.
 2. They browse books and add items to their cart.
 3. When ready, they proceed to checkout, completing the purchase.
 4. The backend processes the order, updates inventory, and sends a confirmation

email (optional).

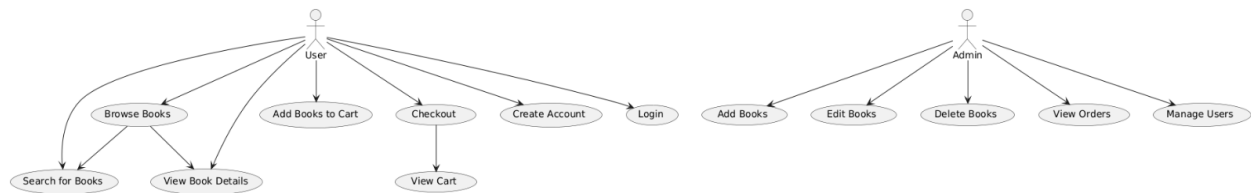
5. Deployment and Hosting:

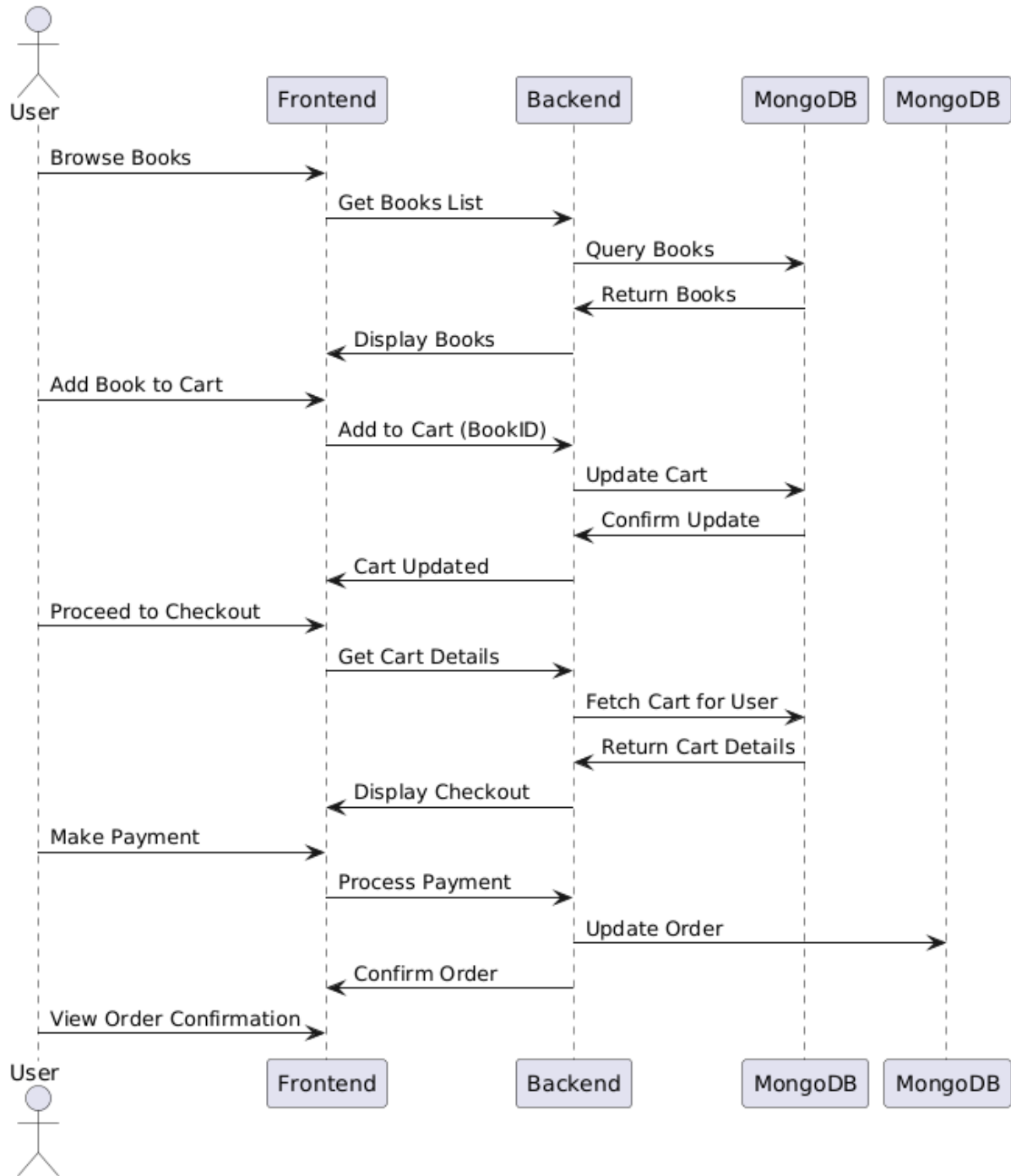
- **Frontend Deployment:** Can be deployed using services like **Netlify** or **Vercel** for seamless CI/CD.
- **Backend Deployment:** Host the backend on platforms like **Heroku** or **DigitalOcean**.
- **Database Deployment:** MongoDB can be hosted on **MongoDB Atlas**, which provides a scalable cloud solution.

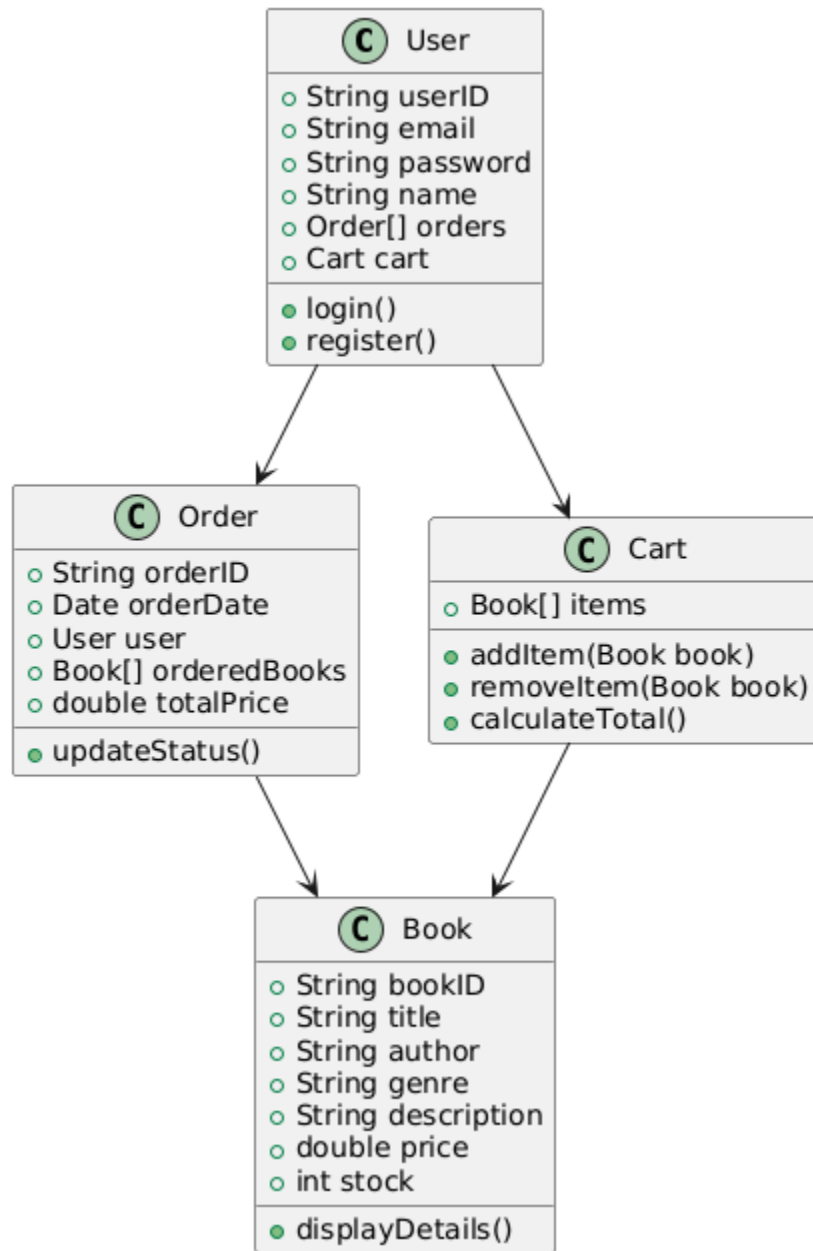
Future Enhancements:

1. **Payment Gateway Integration:** Integrate services like **Stripe** or **PayPal** to handle secure payments.
2. **Recommendation System (Optional):** Implement personalized book suggestions based on user preferences or browsing history.
3. **Multi-language Support:** Expand the platform for global users by adding multi-language and multi-currency features.
4. **Mobile App:** Develop a companion mobile app for better accessibility.

Solution Architecture Diagram:







- **Sprint:** A fixed timebox (5 days) to accomplish defined tasks.
- **Epic:** A large feature or goal broken down into manageable **stories**.
- **Story:** A small unit of work with assigned **story points** (effort estimation).
- **Story Points (Effort):**
 - 1 = Very Easy
 - 2 = Easy
 - 3 = Moderate
 - 5 = Difficult

☐ **Sprint 1: Frontend Setup & UI (Duration: 5 Days)**

☐ **EPIC: Frontend Environment Setup & Basic UI Design**

Day	Task	Story Points
Day 1	Initialize GitHub Repositories & Project Setup	1
	Install Dependencies (React, Express, MongoDB)	1
Day 2	Create Folder Structure for Client & Server	1
	Configure React App and Tailwind CSS (if any)	1
Day 3	Design & Implement Header, Footer	2
Day 4	Develop Book Listing Page	2
Day 5	Test Frontend Components	1

☐ **Sprint 1 Total Story Points: 8**

☐ **Sprint 2: Backend & Authentication (Duration: 5 Days)**

☐ **EPIC: Backend API Development & Auth System**

Day	Task	Story Points
Day 1	Create MongoDB Book Model & Schema	2
	Connect MongoDB to Express Backend	1
Day 2	Build REST APIs (GET, POST, PUT, DELETE) for Books	3
Day 3	Design & Implement User Registration API	2
	Design Login API with JWT Auth	2
Day 4	Create Protected Routes for Admins	3
Day 5	Integrate Auth APIs into Frontend	3

☐ **Sprint 2 Total Story Points: 16**

Team Velocity

- **Total Story Points** = Sprint 1 (8) + Sprint 2 (16) = **24**
- **Number of Sprints** = 2
- **Velocity** = Total Points / Sprints = **24 / 2 = 12 story points/sprint**

□ Summary

- Your team completes **12 Story Points per Sprint** on average.
- Future sprints (e.g., cart, checkout, admin dashboard, search filter) can be planned based on this velocity.

Project Planning Phase

Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)

Date	25 MARCH 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	5 Marks

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Registration	USN-1	As a user, I can register for the application by entering my email, password, and confirming my password.	2	High	Fanish Kumar Diwan
Sprint-1	Registration	USN-2	As a user, I will receive a confirmation email once I have registered for the application.	1	High	Fanish Kumar Diwan
Sprint-2	Registration	USN-3	As a user, I can register for the application through Facebook.	2	Low	Fanish Kumar Diwan
Sprint-1	Registration	USN-4	As a user, I can register for the application through Gmail.	2	Medium	Fanish Kumar Diwan
Sprint-1	Login	USN-5	As a user, I can log into the application by entering email & password.	1	High	Fanish Kumar Diwan
Sprint-2	Login	USN-6	As a user, I can reset my password through a password reset link.	2	Medium	Fanish Kumar Diwan

Sprint-2	Dashboard	USN-7	As a user, I can view a list of all books available in the store.	2	High	Fanish Kumar Diwan
Sprint-2	Dashboard	USN-8	As a user, I can search and filter books based on categories, price, and author.	3	High	Fanish Kumar Diwan
Sprint-3	Cart	USN-9	As a user, I can add selected books to my shopping cart.	2	High	Fanish Kumar Diwan
Sprint-3	Cart	USN-10	As a user, I can remove books from my cart.	1	Medium	Fanish Kumar Diwan
Sprint-3	Checkout	USN-11	As a user, I can checkout and place an order using my details.	3	High	Fanish Kumar Diwan
Sprint-4	Admin Dashboard	USN-12	As an admin, I can add, update, or delete books from the store.	3	High	Fanish Kumar Diwan
Sprint-4	Admin Dashboard	USN-13	As an admin, I can manage user roles and permissions.	3	Medium	Fanish Kumar Diwan

Project Tracker, Velocity & Burndown Chart: (4 Marks)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	20	6 Days	27 April 2025	1 March 2025	20	2 March 2025
Sprint-2	20	6 Days	1 March 2025	2 March 2025	18	2 March 2025
Sprint-3	20	6 Days	2 March 2025	3 March 2025	15	3 March 2025

Sprint-4	20	6 Days	3 March 2025	4 March 2025	20	5 March 2025
-----------------	-----------	---------------	---------------------	---------------------	-----------	---------------------

Velocity:

Imagine we have a 10-day sprint duration, and the velocity of the team is 20 (points per sprint). Let's calculate the team's average velocity (AV) per iteration unit (story points per day)

$$AV = \frac{\text{sprint duration}}{\text{velocity}} = \frac{20}{10} = 2$$

Calculation:

1. **Total Story Points Completed:** This will depend on how many story points are completed in total after several sprints.
 - . For example, if **2 sprints** have been completed and each sprint had 20 points, then the **Total Story Points Completed** = 20 + 20 = 40 story points.
2. **Number of Sprints:** Here it's **2 sprints**.

Now, applying the formula:

Average Velocity (AV) = 40 Story Points/2 Sprints =20 Story Point per Sprint

Project Design Phase-II

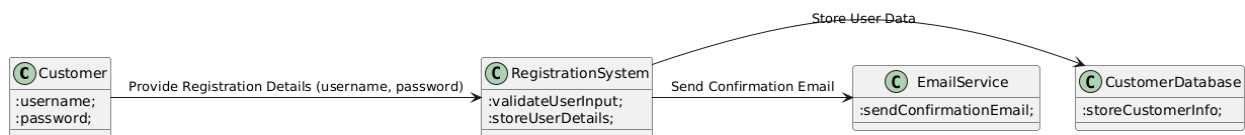
Data Flow Diagram & User Stories

Date	26 March 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	4 Marks

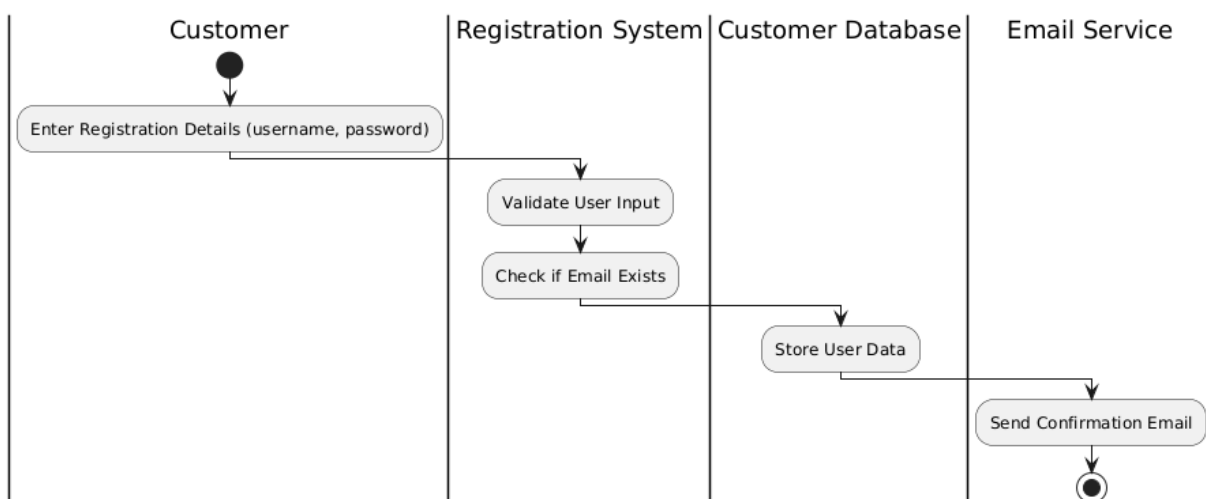
Data Flow Diagrams:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.

DFD Level 0 (Industry Standard)



FLOW Diagram



User Stories Table

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance Criteria	Priority	Release
Customer (Mobile User)	Registration	USN-1	As a user, I can register for the application by entering my email, password, and confirming my password.	I can access my account/dashboard after successful registration.	High	Sprint-1
Customer (Mobile User)	Registration	USN-2	As a user, I will receive a confirmation email once I have registered for the application.	I can receive a confirmation email and click confirm to verify.	High	Sprint-1
Customer (Mobile User)	Registration	USN-3	As a user, I can register for the application through Facebook.	I can register & access the dashboard with Facebook Login.	Low	Sprint-2
Customer	Registration	USN-4	As a user, I	I can register &	Medium	Sprint-

(Mobile User)			can register for the application through Gmail.	access the dashboard with Gmail Login.	m	1
Customer (Mobile User)	Login	USN-5	As a user, I can log into the application by entering my email & password.	I can successfully login to my account after providing valid details.	High	Sprint-1
Customer (Web User)	Dashboard	USN-6	As a web user, I can view my personal dashboard displaying my orders, wish list, and recommended books.	The dashboard displays my personal data (orders, wish list, etc.)	High	Sprint-1
Customer (Web User)	Dashboard	USN-7	As a web user, I can search for books using filters such as genre, price range, and author.	I can filter books based on criteria such as genre, price, etc.	High	Sprint-2
Customer Care Executive	User Management	USN-8	As a customer care executive, I can view customer details and assist with account issues or queries.	I can access user details and provide support as required.	Medium	Sprint-2

Customer Care Executive	Order Management	USN-9	As a customer care executive, I can view customer orders and assist in tracking or modifying them.	I can search, view, and track customer orders in the system.	Medium	Sprint-2
Administrator	Admin Dashboard	USN-10	As an administrator, I can manage users, products, and view reports for sales and user activity.	I can manage user accounts, products, and view analytics.	High	Sprint-3
Administrator	Product Management	USN-11	As an administrator, I can add, update, or delete products from the catalog.	I can modify the product catalog by adding, updating, or removing products.	High	Sprint - 3

Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	27 MARCH 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	4 Marks

Functional Requirements

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Registration	Registration through Form
		Registration through Gmail
FR-2	User Confirmation	Confirmation via Email
		Confirmation via OTP
FR-3	User Login	Login using Email & Password
FR-4	Product Management	Add new products to the catalog
		Update product details (price, description, availability)
		Delete products from the catalog
FR-5	Search and Filter	Search for books by genre, price range, author, and ratings
		Apply filters for sorting books (e.g., by price, author, or rating)
FR-6	Order Management	Place an order for selected products
		Track order status
FR-7	User Dashboard	View order history
		View and manage wish list
FR-8	Payment Integration	Process payments using integrated bank transfer or COD (Cash on Delivery)
FR-9	Admin Panel	Manage users (add, update, delete)
		View sales analytics

Non-Functional Requirements

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	The application should be user-friendly and easy to navigate. Users should be able to easily complete tasks without confusion.
NFR-2	Security	The application should ensure data protection and secure communication (e.g., SSL encryption, password hashing, OTP).
NFR-3	Reliability	The application should be reliable and stable, with minimal downtime or errors. It should work as expected across different devices.
NFR-4	Performance	The application should load within 3 seconds and support smooth user interactions even under heavy load.
NFR-5	Availability	The application should be available 99.9% of the time (uptime).
NFR-6	Scalability	The application should be scalable to handle an increasing number of users and products without compromising performance.

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	28 March 2025
Team ID	SWTID1744544274
Project Name	Book-Store
Maximum Marks	4 Marks

Technical Architecture:

The **Book Store** project employs a modern, scalable architecture designed to provide a seamless experience for both users and administrators. The application is structured to handle user interactions through a **React.js** frontend, which offers a dynamic and responsive interface for browsing and purchasing books. The frontend communicates with a **Node.js** backend, built using the **Express.js** framework, which handles core business logic like user authentication, product management, and order processing.

S.No	Component	Description	Technology
1	User Interface	How user interacts with the application. Web-based interface for browsing and purchasing books.	HTML, CSS, JavaScript, React.js
2	Backend Application Logic	Handles business logic for user registration, login, product management, etc.	Node.js, Express.js
3	Database	Stores user, product, and order information.	MongoDB (NoSQL database)
4	Authentication	Manages user authentication and sessions.	JWT (JSON Web Tokens)
5	Payment Gateway	Processes payments for book purchases.	Stripe API
6	External API-1	Fetches book data from external services.	Google Books API
7	Cloud Storage	Stores book images and other static content.	Cloud storage (e.g., Google Cloud Storage)
	Server	Hosts the application and handles the	Local servers or any

8	Infrastructure	traffic.	preferred cloud service

Table 2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	List the open-source frameworks used.	React.js, Node.js, Express.js, MongoDB
2	Security Implementations	Security controls to ensure safe login and transactions.	JWT for authentication, HTTPS, bcrypt for password hashing
3	Scalable Architecture	Justification for scalability. Scalable for handling multiple users and transactions.	Microservices architecture, horizontal scaling with cloud hosting
4	Availability	Ensures high availability for users.	Load balancing, CDN, replication in database
5	Performance	Application designed to handle multiple requests and serve a large catalog of books.	Caching, CDNs for static content, server load balancing

Book Store Project Report

1. INTRODUCTION

1.1 Project Overview

The Online Book Store is a web-based MERN stack project designed to provide users with a seamless platform to browse, search, and purchase books. The application aims to deliver a user-friendly and responsive interface for students, professionals, and book lovers.

1.2 Purpose

The purpose of this project is to develop a full-stack book store using modern technologies that address user needs, improve the shopping experience, and provide administrative control over inventory and user management.

2. IDEATION PHASE

2.1 Problem Statement

Users struggle to find, compare, and purchase books online efficiently due to a lack of personalized, secure, and user-friendly platforms.

2.2 Empathy Map Canvas

Empathy Map Canvas created to understand users:

- Thinks: Needs affordable and trustworthy book options.
- Sees: Cluttered UI, limited filters, outdated design.
- Says: "Why is it so hard to find the right book?"
- Hears: Recommendations from peers, YouTube influencers.
- Feels: Frustrated with navigation, excited for good deals.
- Does: Searches books, reads reviews, adds to cart.
- Pains: Unclear delivery time, poor payment options.
- Gains: Fast search, secure payment, book previews.

2.3 Brainstorming

Ideas were generated for user experience, backend, frontend, and feature enhancements including: wishlist, user reviews, responsive design, admin dashboard, secure payments, and book recommendations.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

A visual map showing steps from book search to purchase and feedback.

3.2 Solution Requirement

Functional Requirements:

- User Registration/Login
- Book Listing & Filtering
- Add to Cart & Checkout
- Admin Book Management

Non-Functional Requirements:

- Secure Authentication
- Responsive UI
- Fast Loading Speed

3.3 Data Flow Diagram

Data flow diagram (DFD) will illustrate user interactions with the system components.

3.4 Technology Stack

- Frontend: React.js, Tailwind CSS
- Backend: Node.js, Express.js
- Database: MongoDB
- Authentication: JWT
- Image Hosting: Cloudinary/Firebase
- Payment Gateway: Stripe/Razorpay

4. PROJECT DESIGN

4.1 Problem Solution Fit

The project aligns the user's problem of finding reliable book platforms with a custom-built solution focused on usability, accessibility, and feature richness.

4.2 Proposed Solution

Develop a responsive, feature-rich online book store that supports real-time inventory, secure payments, and personalized recommendations.

4.3 Solution Architecture

The architecture follows the MERN stack model with RESTful APIs for frontend-backend communication.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The development was split into sprints: UI, API, Auth, Cart, Admin Panel, Deployment.

6. FUNCTIONAL AND PERFORMANCE TESTING

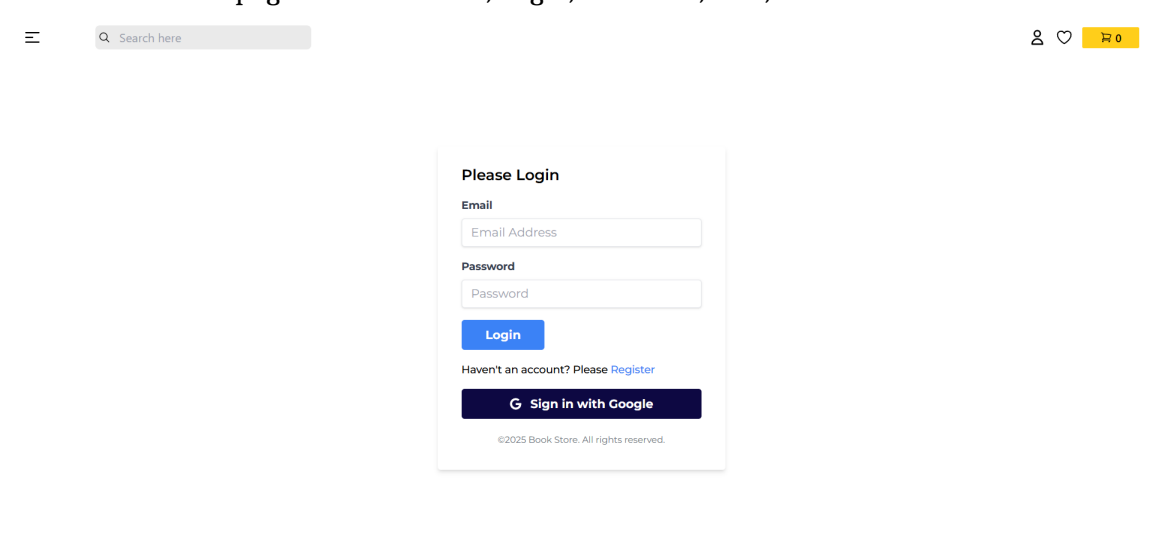
6.1 Performance Testing

Testing conducted using tools like Postman, Lighthouse for API & page performance.

7. RESULTS

7.1 Output Screenshots

Screenshots of UI pages such as Home, Login, Book List, Cart, and Admin Panel.





Search here



0

Please Login

Email

Email Address

Password

Password

Login

Haven't an account? Please [Register](#)

 Sign in with Google

©2025 Book Store. All rights reserved.



Search here



0

Shopping cart

Clear Cart

No product found!

Subtotal

Shipping and taxes calculated at checkout.

\$0.00

Checkout

or [Continue Shopping](#) →



Search here



0

Please Login

Email

Email Address

Password

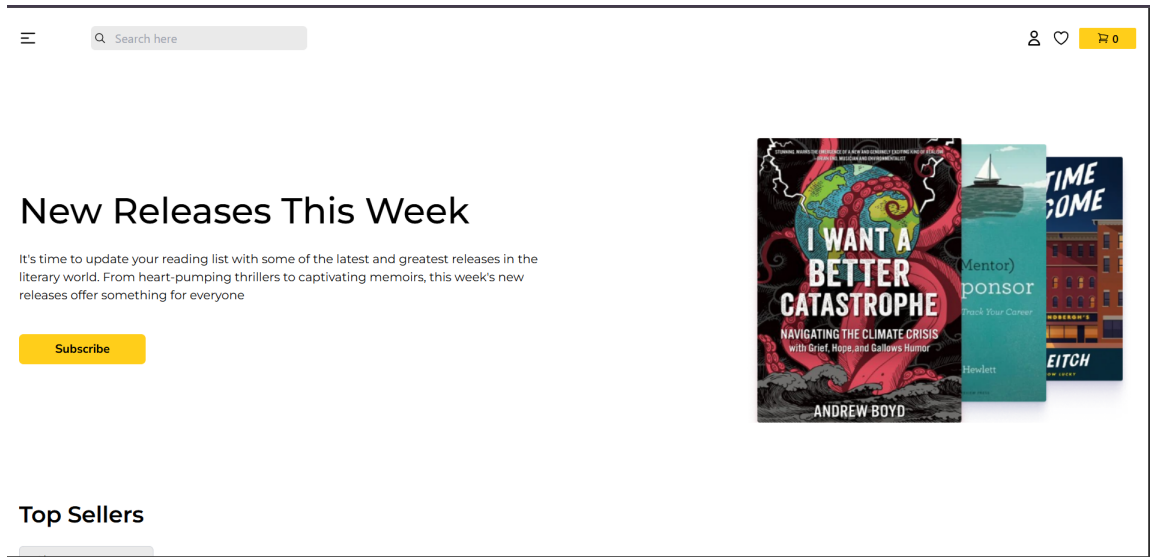
Password

Login

Haven't an account? Please [Register](#)

 Sign in with Google

©2025 Book Store. All rights reserved.



8. ADVANTAGES & DISADVANTAGES

Advantages:

- Modern UI
- Easy navigation
- Secure authentication

Disadvantages:

- No offline support
- Initial hosting cost

9. CONCLUSION

The Online Book Store project provides a comprehensive solution to current e-commerce challenges in book retail. It simplifies the user's buying journey while offering administrative control and future scalability.

10. FUTURE SCOPE

- AI-based recommendations
- Voice-based search
- Multilingual interface
- Mobile app integration

11. APPENDIX

Source Code: <https://github.com/fanishkdiwan/Book-store-MERN.git>

Project Documentation: Full Stack Development with MERN

1. Introduction

Project Title:

Book-Store Web Application

Team Members:

- **Member 1:** [Full Stack Developer] – Fanish Kumar Diwan

2. Project Overview

Purpose:

The purpose of this project is to create a fully functional Book Store Web Application using the MERN stack. The application will allow users to browse, purchase, and manage books. The goal is to provide an easy-to-use interface for customers, a secure and efficient system for managing inventory, and a user-friendly admin dashboard for managing users and books.

Features:

- **User Registration & Login:** Allow users to register and log in to access the store.
- **Book Management:** Users can browse, search, and filter books by category, price, and author.
- **Shopping Cart:** Users can add books to the cart and proceed to checkout.
- **Order Management:** View past orders and track deliveries.
- **Admin Panel:** Admin can manage books, orders, and user information.
- **Authentication:** Secure login with JWT tokens for session management.

3. Architecture

Frontend:

The frontend is built using **React.js**, a JavaScript library for building user interfaces. It is component-based, allowing for a modular, reusable code structure. We use React Router for

navigation and Redux for state management.

Backend:

The backend is powered by **Node.js** and **Express.js**, providing a RESTful API for interacting with the database and managing user requests. **JWT** (JSON Web Tokens) are used for secure user authentication.

Database:

The application uses **MongoDB**, a NoSQL database, to store book data, user information, and order details. The database schema includes collections for users, books, orders, and cart items.

4. Setup Instructions

Prerequisites:

- Node.js (v14 or higher)
- MongoDB (local or Atlas)
- npm (Node Package Manager)

Installation:

1. Clone the repository from GitHub:

```
git clone https://github.com/fanishkdiwan/Book-store-MERN.git
```

2. Navigate to the project directory:

```
cd build-full-stack-book-store-mern-app
```

3. Install dependencies for both the client and the server:

- For **Client** (React frontend):

```
npm install
```

- For **Server** (Node.js backend):

```
npm install
```

4. Set up environment variables:

- In the **client** and **server** directories, create `.env` files and configure the necessary variables such as the MongoDB URI, JWT secret, etc.

5. Folder Structure

Client:

- **/src:** Contains all React components and pages.
- **/components:** Reusable components like buttons, forms, etc.
- **/pages:** Pages such as Home, BookDetail, Cart, Checkout, etc.
- **/redux:** Redux store and reducers for state management.

Server:

- **/models:** Contains MongoDB models for books, users, and orders.
- **/routes:** API routes for handling requests.
- **/controllers:** Business logic for processing data from requests.
- **/middleware:** Authentication and authorization middleware.

6. Running the Application

To run the application locally:

1. Start the frontend (client):

```
npm start
```

2. Start the backend (server):

```
npm start
```

This will run the frontend on `http://localhost:3000` and the backend on `http://localhost:5000` by default.

7. API Documentation

1. Register User

- **Endpoint:** `POST /api/users/register`
- **Parameters:** name, email, password
- **Response:** Success message and token.

2. Login User

- **Endpoint:** `POST /api/users/login`

- **Parameters:** email, password
- **Response:** JWT token on successful login.

3. Get All Books

- **Endpoint:** GET /api/books
- **Response:** List of all books.

4. Add Book (Admin)

- **Endpoint:** POST /api/books
- **Parameters:** title, author, price, category, image_url
- **Response:** Success message.

8. Authentication

Authentication is handled using . On successful login, a token is generated and sent to the user. This token must be included in the headers of subsequent requests for authenticated routes.

9. User Interface

Screenshots:

Include the following screenshots or GIFs of key UI features:

1. **Home Page** showing the list of books.
2. **Book Detail Page** with more information about a book.
3. **Cart Page** where users can review their cart.
4. **Admin Dashboard** for managing books and users.

10. Testing

The application was tested using **Jest** for unit testing of React components and **Mocha/Chai** for testing the backend API.

11. Screenshots or Demo

- Screenshots of key pages (Home, Book Details, Cart, Admin Dashboard, etc.) can be added.



Q Search here



0

Please Login

Email

Email Address

Password

Password

Login

Haven't an account? Please [Register](#)

 Sign in with Google

©2025 Book Store. All rights reserved.



Q Search here



0

Please Login

Email

Email Address

Password

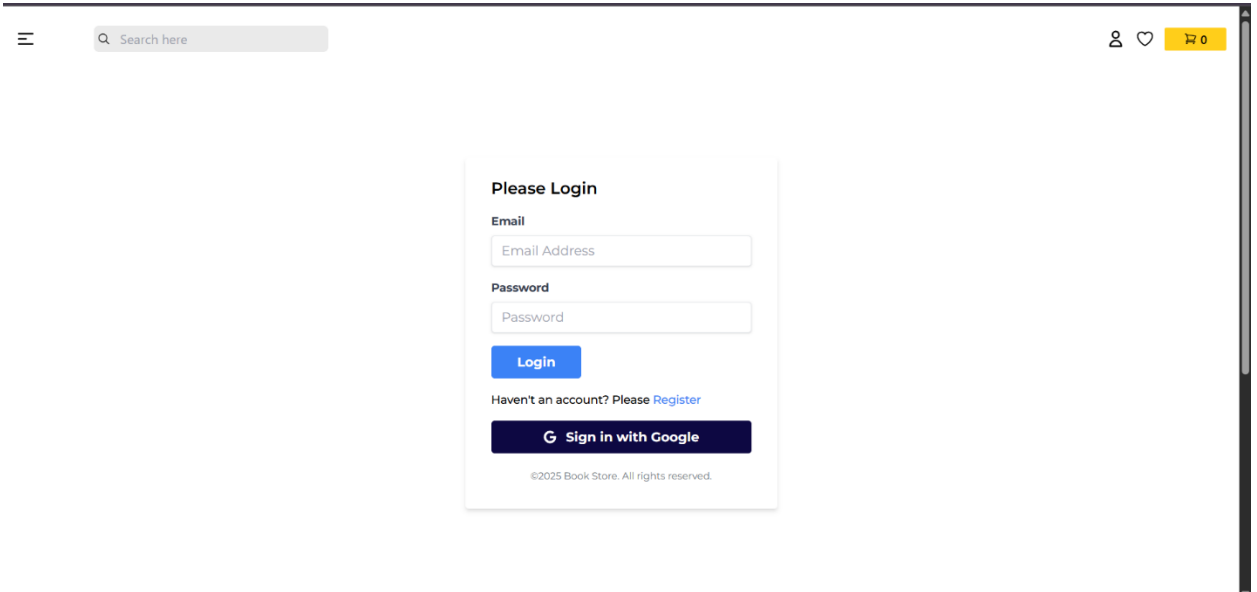
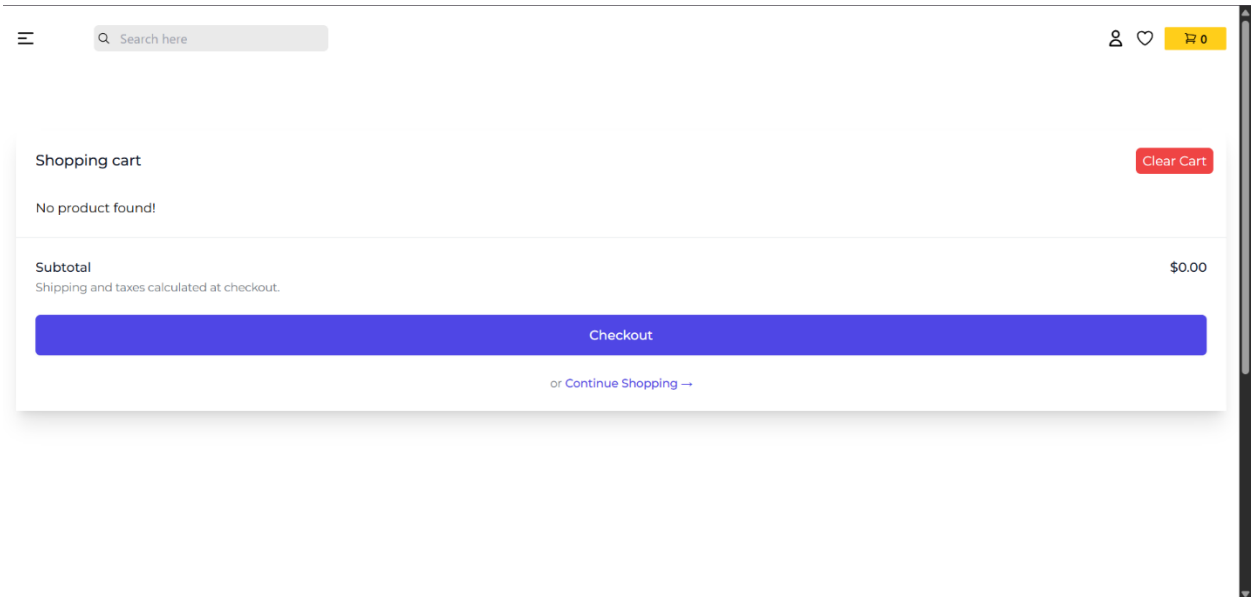
Password

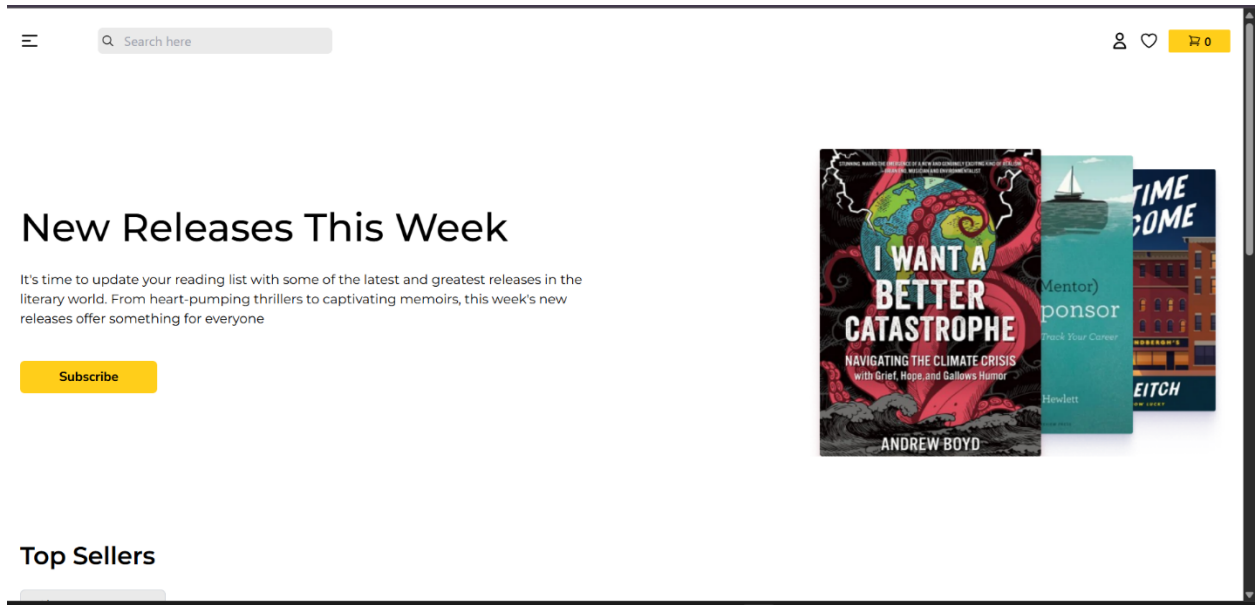
Login

Haven't an account? Please [Register](#)

 Sign in with Google

©2025 Book Store. All rights reserved.





12. Known Issues

- There may be occasional delays in order processing, which is under investigation.
- Currently, user password reset is not implemented.

13. Future Enhancements

- **Product Search:** Implement a search bar to allow users to search for books by title, author, or category.
- **Payment Gateway Integration:** Integrate PayPal or Stripe for real-time payments.
- **Rating System:** Allow users to rate and review books.
- **Admin Roles:** Add more granular roles for admin users to manage orders and users.